Task Scheduler

1. Scheduler Overview

The FreeRTOS scheduler follows a **preemptive + same-priority time-slicing** strategy:

- **Preemptive**: When a higher-priority task is ready, it immediately preempts lower-priority tasks
- **Time-slicing**: Tasks of the same priority take turns sharing CPU time slices (default 1ms/tick)

Importance: The scheduler is a core component of RTOS. Its scheduling strategy directly affects the real-time response capability and multi-tasking efficiency of embedded systems. Proper task priority allocation determines the system's real-time performance, stability, and power consumption. This tutorial uses WS2812 LED color changes to help developers intuitively understand FreeRTOS scheduler behavior, laying the foundation for designing complex multi-tasking systems.

Application Scenarios:

Scheduling Mechanism	Typical Application	Scenario Description
Preemptive Scheduling (High preempts Low)	Emergency motor braking, overcurrent protection	Fault signals must respond immediately, cannot be blocked by lower-priority tasks. Preemption ensures microsecond-level latency

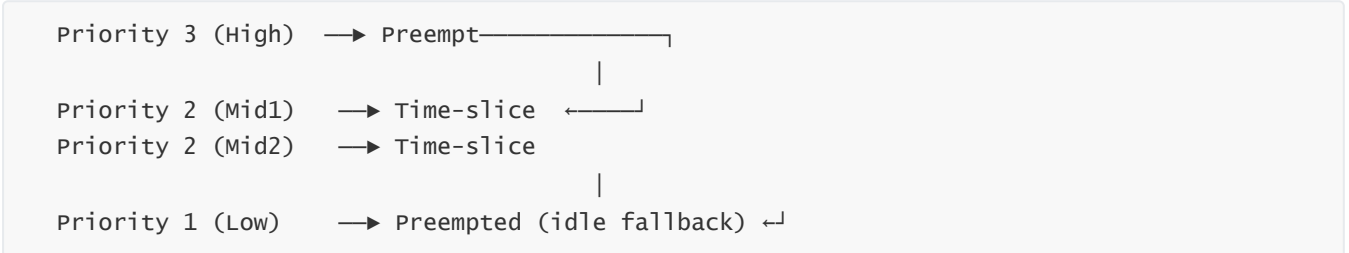
Scheduling Mechanism	Typical Application	Scenario Description
	Wireless communication protocol stack (Wi-Fi/BLE)	Protocol stack timing is strict; packet TX/RX processing has higher priority than UI to avoid packet loss or disconnection
	Sensor-triggered sampling	External interrupt-triggered acquisition tasks must execute immediately to prevent data loss or missed sampling windows
Time-slicing (Same priority round-robin)	Multi-channel sensor polling	Temperature, humidity, pressure, light sensors are equal in priority; no preemption needed, just take turns
	Multi-channel LED animation control	Multiple RGB effects run independently; time-slicing gives each channel equal refresh opportunities
	Log output and status reporting	Debug logs, status LEDs, serial reports and other auxiliary functions use same-priority round-robin, non-blocking
Idle Task (Lowest priority fallback)	System sleep/low-power management	Enter idle task when no urgent tasks; calculate CPU utilization or trigger sleep mode to reduce power consumption
	Background statistics refresh	Update runtime statistics, memory usage and other monitoring data during idle cycles without affecting foreground tasks

This project uses 4 tasks (High/Mid1/Mid2/Low) + 3 GPIO signals (KEY/GPIO2/GPIO4) to visually demonstrate the scheduler's preemption and time-slicing behavior through WS2812 LED color changes.

---

2. Core Concepts

2.1 Priority Scheduling



Rules:

1. Higher priority task ready → Immediately preempts lower priority task
2. Same priority tasks ready → Kernel automatically time-slices (one context switch per tick)

3. Lower priority task runs only when no higher priority task is ready

2.2 Time-Slicing

Two same-priority tasks (Mid1, Mid2) share the CPU; the kernel switches between them per tick:

```
  Mid1: ████............████............████
  Mid2: ....████............████............
        ├──── 1 tick ────┤
```

At the end of each tick, the kernel checks if other same-priority tasks are ready; if so, it switches.

2.3 Task Priority API

```
#define HIGH_PRIO  3   // Highest — KEY press preempts everything
#define MID_PRIO   2   // Medium — GPIO4/GPIO2 trigger execution
#define LOW_PRIO   1   // Lowest, fallback

// Set priority at creation
xTaskCreatePinnedToCore(v_high_task, "High", 2048, NULL, HIGH_PRIO, NULL, 1);
```

---

3. Project Architecture and Flow

```
app_main()
  ├─ gpio_init_all()     ← GPIO0 (pull-up), GPIO2 (pull-up), GPIO4 (pull-down)
  ├─ rgb_init()          ← WS2812 (GPIO48, RMT)
  ├─ xTaskCreate(High, Prio=3, Core 1)   ← Preemption core
  ├─ xTaskCreate(Mid1, Prio=2, Core 1)  ← Time-slice (same priority as Mid2)
  ├─ xTaskCreate(Mid2, Prio=2, Core 1)  ← Time-slice (same priority as Mid1)
  └─ xTaskCreate(Low,  Prio=1, Core 1)  ← Idle fallback

All tasks pinned to Core 1 — single-core scheduling demonstration
```

Scheduling Rules and LED Color Mapping:

Condition	Active Task	LED Color	Scheduling Principle
KEY pressed (GPIO0→L)	High (Prio 3)	**Red**	Preemption — highest priority preempts all lower priorities
GPIO4 connected to 3.3V	Mid1 (Prio 2)	**Purple**	Time-slice — round-robin with Mid2 (same priority)
GPIO2 connected to GND	Mid2 (Prio 2)	**Yellow**	Time-slice — round-robin with Mid1 (same priority)
No trigger	Low (Prio 1)	**Cyan**	Idle fallback — lowest priority, runs when no other tasks ready

4. Hardware Dependencies

Pin	Function	Default Level	Description
GPIO 0 (KEY)	High task trigger	Pull-up → High	Connect to GND (press) → triggers High, red LED
GPIO 2	Mid2 task trigger	Pull-up → High	Connect to GND → triggers Mid2, yellow LED
GPIO 4	Mid1 task trigger	Pull-down → Low	Connect to 3.3V → triggers Mid1, purple LED
GPIO 48	WS2812 RGB LED	—	Displays color corresponding to currently active task

Required Components: `freertos/FreeRTOS`, `freertos/task`, `driver/gpio`, `led_strip`, `esp_log`

This project uses a **third-party library** again: For details, refer to (**Third-party library installation: 2.Basic Peripherals → 10. RGB LED Effects**)

> **Note**: `led_strip` is a third-party component provided by the ESP-IDF Component Registry, not built into IDF. Before using, declare the dependency in `main/idf_component.yml`, otherwise compilation will fail to find the header:
>
> ```
> ## IDF Component Manager Manifest File
> dependencies:
> idf:
>  version: '>=4.1.0'
> espressif/led_strip: ^3.0.3
> ```
>
> After declaration, running `idf.py build` will automatically download the component to `managed_components/`.

5. Source Code Analysis

Complete source: `Source Code → ESP-IDF → 4.FreeRTOS → 4. RTOS_SCHEDULE`

5.1 Header Files and Macro Configuration

The following includes headers, macro definitions, and inline helper functions for GPIO pin mapping, task priority configuration, and polling intervals.

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_log.h"
#include "driver/gpio.h"
#include "led_strip.h"
```

```c
static const char *TAG = "SCHEDULE";

/* GPIO pin definitions */
#define KEY_GPIO       0
#define GPIO2_MID2     2
#define GPIO4_MID1     4
#define RGB_GPIO       48    // WS2812 data pin

/* Task priority — higher number = higher priority */
#define HIGH_PRIO      3     // High task priority
#define MID_PRIO       2
#define LOW_PRIO       1     // Low task priority

/* Polling intervals */
// Active: 1 tick fast loop
#define POLL_TICK_ACTIVE   1
// Key-release debounce: 50ms
#define POLL_MS_POST      50
#define POLL_MS_IDLE     500  // Idle: 500ms polling
// Post-release buffer duration
#define POLL_POST_TICKS    4
static led_strip_handle_t led_strip;

/* GPIO inline helper functions */
static inline bool key_pressed(void) { return gpio_get_level(KEY_GPIO)  == 0; }
static inline bool gpio4_high(void)  { return gpio_get_level(GPIO4_MID1) == 1; }
static inline bool gpio2_low(void)   { return gpio_get_level(GPIO2_MID2) == 0; }
```

5.2 gpio_init_all() — GPIO Initialization

GPIO initialization configures three input pins: KEY pull-up (low when pressed), GPIO2 pull-up (high when floating), GPIO4 pull-down (low when floating). The differentiated pull-up/pull-down configuration ensures floating state doesn't trigger anything by default.

```c
static void gpio_init_all(void)
{
    gpio_config_t key_cfg = {
        .pin_bit_mask = 1ULL << KEY_GPIO,
        .mode         = GPIO_MODE_INPUT,
        .pull_up_en   = GPIO_PULLUP_ENABLE,
        .pull_down_en = GPIO_PULLDOWN_DISABLE,
        .intr_type    = GPIO_INTR_DISABLE,
    };
    gpio_config(&key_cfg);
    gpio_config_t gpio2_cfg = {
        .pin_bit_mask = 1ULL << GPIO2_MID2,
        .mode         = GPIO_MODE_INPUT,
        .pull_up_en   = GPIO_PULLUP_ENABLE,
        .pull_down_en = GPIO_PULLDOWN_DISABLE,
        .intr_type    = GPIO_INTR_DISABLE,
    };
```

```
    gpio_config(&gpio2_cfg);
    gpio_config_t gpio4_cfg = {
        .pin_bit_mask = 1ULL << GPIO4_MID1,
        .mode         = GPIO_MODE_INPUT,
        .pull_up_en   = GPIO_PULLUP_DISABLE,
        .pull_down_en = GPIO_PULLDOWN_ENABLE,
        .intr_type    = GPIO_INTR_DISABLE,
    };
    gpio_config(&gpio4_cfg);
}
```

> **Pull-up vs Pull-down**: GPIO0/GPIO2 pull-up = floating reads high; GPIO4 pull-down = floating reads low. When floating, none of the three conditions are met → Low task is the fallback (cyan).

5.3 v_high_task() — High Priority Task (Prio 3)

Highest priority task; when KEY is pressed, immediately preempts all lower priority tasks and lights the red LED. Active state uses 1-tick fast loop to continuously hold the CPU. After key release, waits 200ms buffer before switching to idle polling.

```
static void v_high_task(void *pv)
{
    bool in  = false;               // Key state flag
    int  cnt = 0;                    // Key-release debounce counter
    while (1) {
        if (key_pressed()) {         // KEY pressed
            cnt = 0;                 // Reset debounce counter
            if (!in) { ESP_LOGI(TAG, "[High] KEY pressed");  in = true; }
            // Red — high priority preempting
            set_rgb(150, 0, 0);
            // 1 tick fast loop, hold preemption
            vTaskDelay(POLL_TICK_ACTIVE);
        } else {                     // KEY released
            if (in)  { ESP_LOGI(TAG, "[High] KEY released"); in = false; }
            // Post-release buffer window
            if (cnt < POLL_POST_TICKS) {
                cnt++;
                // 50ms fast poll, anti-bounce
                vTaskDelay(pdMS_TO_TICKS(POLL_MS_POST));
            } else {
                // 500ms idle poll, yield CPU
                vTaskDelay(pdMS_TO_TICKS(POLL_MS_IDLE));
            }
        }
    }
}
```

> **Active 1-tick fast loop** ensures High task continuously preempts the CPU while KEY is pressed; lower priority tasks cannot run at all.

5.4 v_mid_task1() — Mid1 Task (Prio 2)

Medium priority task; lights purple LED when GPIO4 is high. Same priority as Mid2; when both are triggered, they alternate via time-slicing.

```
static void v_mid_task1(void *pv)
{
    bool in = false;  // Active state flag
    while (1) {
        if (!key_pressed() && gpio4_high())
        {
            if (!in) { ESP_LOGI(TAG, "[Mid1] GPIO4 HIGH"); in = true; }
            // Purple — Mid1 running
            set_rgb(120, 0, 150);
        } else {
            if (in)  { ESP_LOGI(TAG, "[Mid1] stop"); in = false; }
        }
        vTaskDelay(pdMS_TO_TICKS(POLL_MS_IDLE)); // 500ms polling
    }
}
```

5.5 v_mid_task2() — Mid2 Task (Prio 2)

Medium priority task; lights yellow LED when GPIO2 is low. Same priority as Mid1; when both are triggered, they alternate via time-slicing.

```
static void v_mid_task2(void *pv)
{
    bool in = false;  // Active state flag
    while (1) {
        if (!key_pressed() && gpio2_low())
        {
            if (!in) { ESP_LOGI(TAG, "[Mid2] GPIO2 LOW"); in = true; }
            // Yellow — Mid2 running
            set_rgb(150, 150, 0);
        } else {
            if (in)  { ESP_LOGI(TAG, "[Mid2] stop"); in = false; }
        }
        vTaskDelay(pdMS_TO_TICKS(POLL_MS_IDLE)); // 500ms polling
    }
}
```

5.6 v_low_task() — Low Priority Task (Prio 1)

Lowest priority task; runs only when all high/medium priority trigger conditions are unmet, lighting cyan LED as idle fallback. Can be preempted at any time.

```
static void v_low_task(void *pv)
{
    bool in = false;  // Active state flag
    while (1) {
```

```
        // All idle — no trigger
        if (!key_pressed() && !gpio4_high() && !gpio2_low())
        {
            if (!in) { ESP_LOGI(TAG, "[Low] all idle"); in = true; }
            // Cyan — idle fallback
            set_rgb(0, 150, 150);
        } else {
            if (in)  { ESP_LOGI(TAG, "[Low] preempted"); in = false; }
        }
        vTaskDelay(pdMS_TO_TICKS(POLL_MS_IDLE)); // 500ms polling
    }
}
```

5.7 app_main() — Main Entry

Main entry initializes GPIO and RGB LED, creates four tasks all pinned to Core 1, constructing a scheduling demonstration environment with multi-level preemption and same-priority time-slicing on a single core.

```
void app_main(void)
{
    gpio_init_all();        // Initialize GPIO
    rgb_init();             // Initialize WS2812 RGB LED

    /* Suppress RMT driver error logs (floating LED may trigger) */
    esp_log_level_set("rmt", ESP_LOG_NONE);
    esp_log_level_set("led_strip_rmt", ESP_LOG_NONE);

    /* High(Pri=3): Preemption core */
    xTaskCreatePinnedToCore(v_high_task,  "High", 2048, NULL, HIGH_PRIO, NULL, 1);
    /* Mid1(Pri=2): Time-slice round-robin with Mid2 */
    xTaskCreatePinnedToCore(v_mid_task1, "Mid1", 2048, NULL, MID_PRIO,  NULL, 1);
    /* Mid2(Pri=2): Time-slice round-robin with Mid1 */
    xTaskCreatePinnedToCore(v_mid_task2, "Mid2", 2048, NULL, MID_PRIO,  NULL, 1);
    /* Low(Pri=1): Idle fallback */
    xTaskCreatePinnedToCore(v_low_task,  "Low",  2048, NULL, LOW_PRIO,  NULL, 1);
}
```

6. Operation Procedure

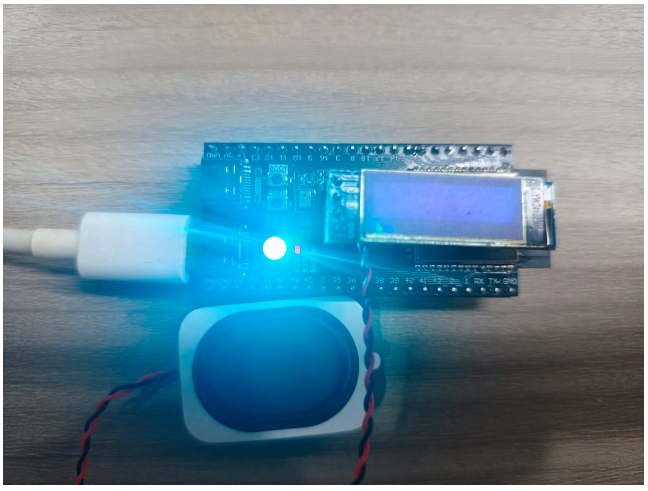
6.1 Build and Flash Procedure

For detailed flash procedure: 1.Development Basics → 3.Build and Flash Procedure → Build and Flash Procedure

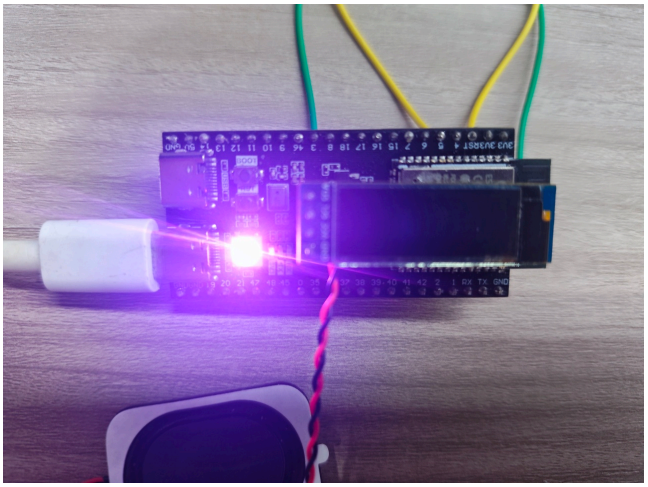
6.2 Normal Operation

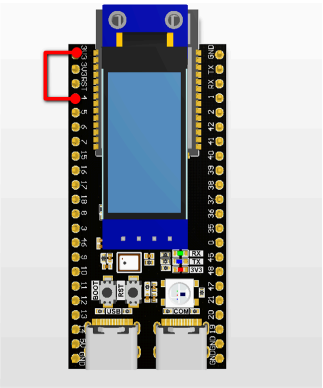
Step	Wiring	LED Color	Description
1	No connections	**Cyan**	All three trigger conditions unmet (KEY not pressed, GPIO4 low, GPIO2 high). Only Low task (Prio 1) is ready; as idle fallback it gets CPU time
2	GPIO4 → 3.3V	**Purple**	GPIO4 high triggers Mid1 task (Prio 2) ready; kernel immediately preempts Low task — higher priority ready directly剥夺低优先级 CPU, preemption mechanism takes effect
3	GPIO2 → GND	**Yellow**	GPIO2 low triggers Mid2 task (Prio 2) ready; preempts Low task (Prio 1). Same as Step 2 for demonstrating preemption of same-priority tasks
4	GPIO4 → 3.3V + GPIO2 → GND	**Purple/Yellow alternating**	Mid1 and Mid2 both ready at same priority (Prio 2); kernel starts time-slicing: switches between them every tick (1ms). LED alternates purple/yellow — demonstrating same-priority time-slicing
5	Press BOOT key (KEY/GPIO0)	**Red**	KEY press triggers High task (Prio 3) ready. Highest priority unconditionally preempts all medium/low tasks. LED stays solid red — proving FreeRTOS always runs the highest priority ready task

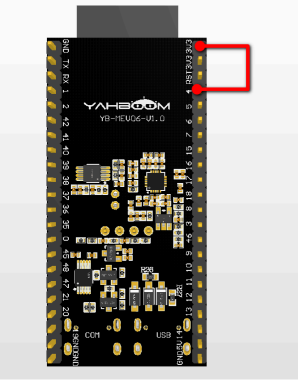
1. With no connections, only Low task (Prio 1) runs: all three trigger conditions (KEY not pressed, GPIO4 low, GPIO2 high) are unmet; High, Mid1, Mid2 are all blocked waiting for triggers; Low task as lowest priority idle fallback gets CPU. LED shows **cyan**.



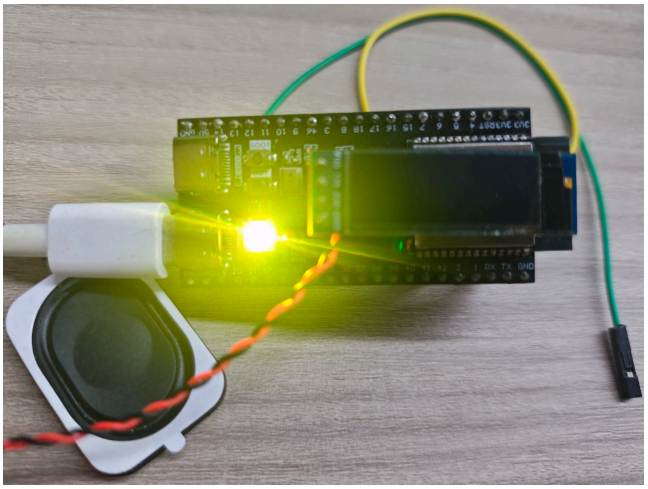
2. GPIO4 → 3.3V → Mid1 task (Prio 2) wakes and becomes ready. Kernel finds a higher priority task than Low (Prio 1) in the ready queue, immediately switches CPU from Low to Mid1. Serial prints `[Mid1] GPIO4 HIGH`. LED shows **purple**. This step demonstrates **preemptive scheduling**: once a higher priority task is ready, it immediately剥夺 lower priority tasks' CPU.

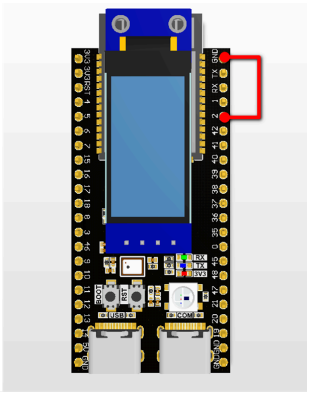


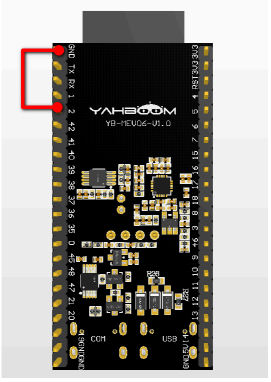




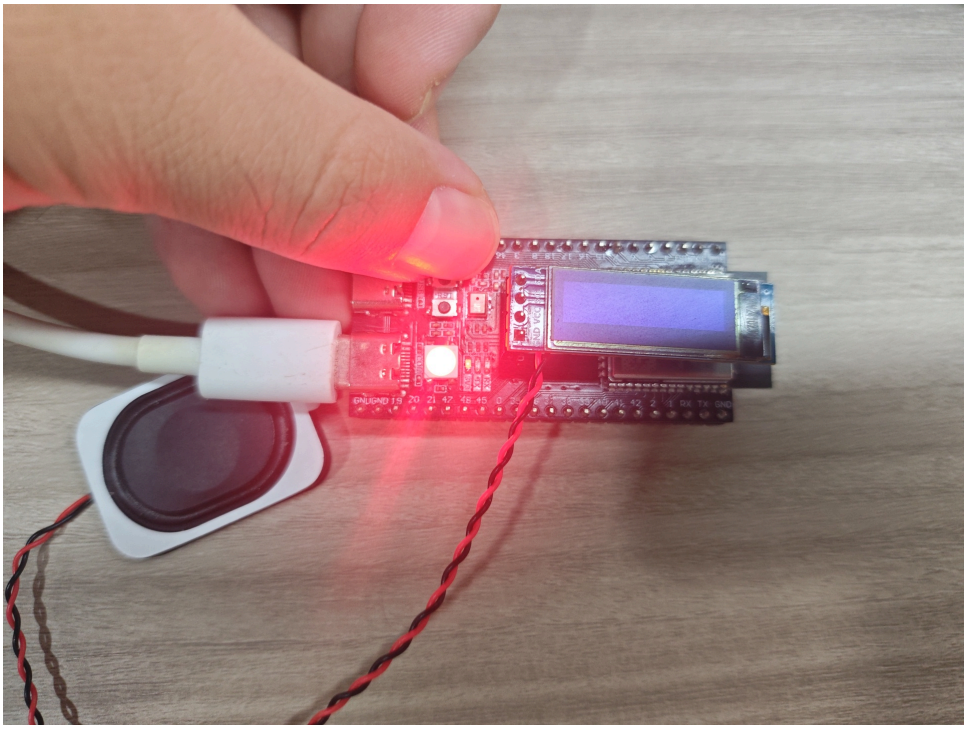
3. GPIO2 → GND → Mid2 task (Prio 2) wakes and becomes ready, similarly preempts Low task (Prio 1) to get CPU. Serial prints `[Mid2] GPIO2 LOW`. LED shows **yellow**. Mid2 has same priority as Mid1 (Prio 2), but only Mid2 is triggered in this step, so only yellow shows — same priority doesn't trigger round-robin when only one task is ready.







4. GPIO4 → 3.3V AND GPIO2 → GND → Mid1 and Mid2 both ready at same priority (Prio 2). Kernel starts **time-slicing**: at each tick (default 1ms) end, checks if other same-priority tasks are ready; if so, forces context switch. Result: two tasks alternate execution; LED **alternates purple and yellow**, visually demonstrating time-division multiplexing of same-priority tasks.
5. When BOOT key is pressed (KEY/GPIO0→low) → High task (Prio 3) becomes ready. As the highest priority in the system (Prio 3 > Prio 2 > Prio 1), regardless of whether Mid1, Mid2 or Low is currently running, kernel unconditionally switches CPU to High task. LED stays solid **red**. Serial prints `[Mid1] stop`, `[Mid2] stop` indicating medium priority tasks were forcibly剥夺. This step proves FreeRTOS's core scheduling rule — **always run the highest priority ready task**.



Log example:

```
I (1263) SCHEDULE: [Low] all idle                    ← Initial: cyan
I (2000) SCHEDULE: [Low] preempted
I (2000) SCHEDULE: [Mid1] GPIO4 HIGH                 ← GPIO4 → 3.3V: purple
I (3000) SCHEDULE: [Mid2] GPIO2 LOW                  ← GPIO2 → GND: yellow
I (3000) SCHEDULE: [Mid1] GPIO4 HIGH                 ← Mid1 and Mid2 alternating
I (4000) SCHEDULE: [High] KEY pressed                ← KEY pressed: red
I (4000) SCHEDULE: [Mid1] stop                       ← Mid1 preempted
I (4000) SCHEDULE: [Mid2] stop                       ← Mid2 preempted
I (5000) SCHEDULE: [High] KEY released               ← KEY released
I (5000) SCHEDULE: [Mid2] GPIO2 LOW                  ← Mid2 resumes
```

---

7. FAQ

Symptom	Cause	Solution
Only red, no other colors	KEY not released / GPIO0 config error	Confirm KEY is high when released
Mid1/Mid2 don't alternate	Same priority but different cores	Ensure both tasks use `xTaskCreatePinnedToCore(..., 1)` on same core
Low task never runs	High/medium priority tasks always triggered	Check GPIO2/GPIO4 default levels
GPIO2 not wired but Mid2 triggers	Pull-down resistor not configured	Configure `pull_up_en` → floating = high ≠ trigger condition (low)
LED blink pattern incorrect	LED operation from concurrent tasks	Design: last task to write LED determines color — higher priority overrides lower